

Guía de Supervivencia Práctica 3 de Python

 Guía de Supervivencia — Práctica 3 de Python

ÍNDICE

1. Cheat Sheet de Sintaxis Básica (Búsqueda Rápida)

Sección 3 — Listas

Sección 4 — Funciones y Paquetes

Sección 5 — NumPy

Sección 6 — Diccionarios y Pandas

Sección 7 — Lógica, Control de Flujo y Filtrado

Sección 8 — Bucles

Guía de Supervivencia — Práctica 3 de Python

Documento pensado para buscar con `Ctrl + F`. Cada bloque de código tiene comentarios en lenguaje coloquial para encontrar rápido lo que necesitas.

ÍNDICE

1. Cheat Sheet de Sintaxis Básica
 2. Sección 3 — Listas (Ejercicios 1-4)
 3. Sección 4 — Funciones y Paquetes (Ejercicios 5-7)
 4. Sección 5 — NumPy (Ejercicios 8-11)
 5. Sección 6 — Diccionarios y Pandas (Ejercicios 12-15)
 6. Sección 7 — Lógica y Control de Flujo (Ejercicios 16-18)
 7. Sección 8 — Bucles (Ejercicios 19-20)
-

1. Cheat Sheet de Sintaxis Básica (Búsqueda Rápida)

🔗 Listas: creación, acceso y slicing

```
# cómo crear una lista
catalogo = ["Auriculares", 59.90, "Tablet", 219.00]

# cómo saber cuántos elementos tiene una lista
len(catalogo)

# cómo acceder a un elemento por índice (empieza en 0)
catalogo[0]      # "Auriculares"
catalogo[-1]     # último elemento

# cómo hacer slicing (trocear una lista): lista[inicio:fin:paso]
catalogo[0:2]    # del 0 al 1 (el "fin" NO se incluye)
catalogo[::2]    # todos los elementos en posiciones PARES (paso 2 desde el principio)
catalogo[1::2]   # todos los elementos en posiciones IMPARES
catalogo[::-1]   # la lista INVERTIDA (paso -1)
catalogo[-3:]    # los últimos 3 elementos

# cómo buscar la posición de un elemento sin escribir el índice a mano
catalogo.index("Tablet")

# cómo recorrer una lista con un bucle for
for elemento in catalogo:
    print(elemento)

# cómo sumar y calcular la media de una lista de números
precios = [59.90, 219.00, 18.50]
total = sum(precios)
media = round(sum(precios) / len(precios), 2)
```

🔗 Copias vs referencias (el error más típico)

```
# cómo NO copiar una lista (esto crea una REFERENCIA, no una copia)
a = [1, 2, 3]
b = a          # b apunta al MISMO objeto que a
b[0] = 99
print(a)       # [99, 2, 3] -> ¡a también cambió!

# cómo copiar una lista DE VERDAD (copia independiente)
copial = list(a)    # con la función list()
copia2 = a[:]       # con slicing completo

# cómo comprobar que son objetos distintos en memoria
id(a) == id(copial) # False -> son independientes

# métodos que modifican la lista EN SITIO (in-place) y devuelven None
lista = [3, 1, 2]
lista.append(4)      # añade un elemento al final
lista.extend([5, 6]) # añade varios elementos al final
lista.insert(1, 99)  # inserta un elemento en una posición concreta
lista.sort()         # ordena la lista ORIGINAL (la modifica)
lista.sort(reverse=True) # ordena de mayor a menor
lista.reverse()      # invierte la lista original

# cómo ordenar SIN modificar la original
nueva = sorted(lista) # devuelve una lista NUEVA ordenada

# cómo eliminar elementos
del lista[0]         # elimina por posición (no devuelve nada)
eliminado = lista.pop() # elimina el ÚLTIMO y lo devuelve
eliminado = lista.pop(0) # elimina el PRIMERO y lo devuelve
```

🔗 Funciones (cómo crear y usar funciones)

```

# cómo definir una función básica
def calcular_importe(unidades, precio_unitario, descuento_pct=0):
    """Devuelve el importe neto redondeado a dos decimales."""
    bruto = unidades * precio_unitario
    return round(bruto * (1 - descuento_pct / 100), 2)

# cómo dar un valor por defecto a un parámetro (así es opcional al llamar)
# "descuento_pct=0" significa que si no lo pasas, vale 0

# cómo llamar a una función pasando argumentos por posición
calcular_importe(3, 59.90, 10)

# cómo llamar a una función usando argumentos por palabra clave (keyword arguments)
calcular_importe(unidades=3, precio_unitario=59.90, descuento_pct=10)

# cómo devolver varios valores a la vez (usando una tupla)
def resumen_pedido(unidades, precio, descuento_pct=0):
    bruto = unidades * precio
    neto = calcular_importe(unidades, precio, descuento_pct)
    ahorro = round(bruto - neto, 2)
    return bruto, ahorro, neto # esto es una tupla (bruto, ahorro, neto)

# cómo "desempaquetar" una tupla en varias variables
importe_bruto, ahorro, importe_netto = resumen_pedido(3, 59.90, 10)

# cómo ver la documentación (docstring) de una función
help(calcular_importe)

```

◆ Cadenas de texto (strings): métodos más usados

```

texto = " H0G-0045-ES "

texto.strip()      # quita espacios al principio y al final
texto.upper()      # pasa todo a MAYÚSCULAS
texto.lower()      # pasa todo a minúsculas
texto.replace("-", ".") # sustituye un carácter/cadena por otro
texto.split("-")    # trocea el string en una lista, usando "-" como separador
texto.count("0")    # cuenta cuántas veces aparece un carácter/substring
texto.index("ES")   # posición donde empieza "ES"

# OJO: estos métodos NO modifican el string original, DEVUELVEN uno nuevo
# los strings en Python son inmutables (como en Scala)

# cómo unir una lista de strings en un único string con un separador
partes = ["ELC", "0012", "ES"]
"-".join(partes)    # "ELC-0012-ES"
"|".join(partes)    # "ELC | 0012 | ES"

```

◆ Importar paquetes y módulos

```
# cómo importar un paquete completo
import math
math.sqrt(2809)      # raíz cuadrada
math.ceil(4.2)       # redondeo hacia arriba

# cómo importar solo lo que necesitas (importación selectiva)
from math import pi, ceil
ceil(1375 / 24)      # ya no hace falta escribir math.ceil

# cómo importar un paquete con un alias (muy típico con numpy y pandas)
import numpy as np
import pandas as pd
np.__version__
pd.__version__

# cómo fijar una semilla aleatoria para que los resultados sean reproducibles
import random
random.seed(42)
random.randint(1, 5)  # siempre dará la misma secuencia con la misma semilla

# cómo importar tus propias funciones desde otro archivo .py
import sys
sys.path.append("../src")      # le dices a Python dónde buscar
from utilidades import calcular_importe, clasificar_ticket
```

◆ NumPy: arrays y operaciones vectorizadas

```

import numpy as np

# cómo crear un array a partir de una lista
arr = np.array([1, 2, 3, 4])

# cómo ver info básica de un array
arr.dtype    # tipo de dato (int64, float64...)
arr.shape    # dimensiones, ej. (4,) o (3, 5)
arr.ndim     # número de dimensiones
arr.size     # número total de elementos

# LA GRAN VENTAJA de NumPy: operar TODO el array de golpe (vectorización)
# esto NO funciona igual con listas normales de Python:
precios = np.array([10, 20, 30])
unidades = np.array([2, 1, 3])
importes = precios * unidades    # multiplica elemento a elemento -> array([20,20,90])

# esto SÍ se puede hacer con listas normales pero NO multiplica elemento a elemento
lista_precios = [10, 20, 30]
lista_precios * 2    # repite la lista dos veces, no multiplica cada elemento

# cómo aplicar un recargo/porcentaje a todo el array de una vez
importes_con_recargo = importes * 1.04    # +4% a todos los elementos

# cómo hacer una máscara booleana (filtrar según una condición)
caros = precios > 15    # array de True/False
precios[caros]          # solo los elementos que cumplen la condición
caros.sum()             # cuenta cuántos True hay (True cuenta como 1)

# cómo combinar varias condiciones en NumPy
# ¡OJO! con arrays de NumPy NO se usa "and"/"or", se usa "&" y "|"
# y cada condición debe ir ENTRE PARÉNTESIS
seleccion = (precios > 15) & (unidades >= 2)
np.where(caros)          # devuelve las POSICIONES que cumplen la condición
np.logical_and(precios > 15, unidades >= 2)    # alternativa a "&"

# cómo crear arrays 2D (matrices)
matriz = np.array([[1, 2, 3], [4, 5, 6]])
matriz.shape    # (2, 3) -> 2 filas, 3 columnas
matriz[0, 1]    # fila 0, columna 1 -> indexación 2D
matriz[:, 0]    # toda la columna 0
matriz[0, :]    # toda la fila 0
matriz.sum(axis=0)    # suma por columnas
matriz.sum(axis=1)    # suma por filas
np.vstack([matriz, [7, 8, 9]])    # apila una fila nueva debajo
np.argmax(matriz.sum(axis=0))    # posición del valor máximo

# cómo generar números aleatorios reproducibles con el generador moderno
rng = np.random.default_rng(2024)
datos = rng.normal(68, 6.5, 500)    # 500 valores con media 68 y desviación 6.5

# estadística descriptiva básica
np.mean(datos)
np.median(datos)
np.std(datos)
np.percentile(datos, 75)
np.corrcoef(datos, otros_datos)    # matriz de correlación

```

♦ Diccionarios (cómo trabajar con `dict`)

```

# cómo crear un diccionario
tarifas = {"Norte": 4.95, "Sur": 5.50, "Este": 5.20}

# cómo acceder a un valor por su clave
tarifas["Norte"]

# cómo añadir o actualizar una clave (si existe la actualiza, si no la crea)
tarifas["Insular"] = 9.80
tarifas["Norte"] = 5.95

# cómo eliminar una clave
del tarifas["Sur"]

# cómo comprobar si una clave existe
"Norte" in tarifas    # True

# cómo recorrer un diccionario (clave y valor a la vez)
for region, tarifa in tarifas.items():
    print(region, "->", tarifa)
# si solo pones "for region in tarifas" recorres SOLO LAS CLAVES

# cómo crear un diccionario anidado (diccionario dentro de diccionario)
red = {
    "Norte": {"almacen": "ALM-NORTE", "tarifa": 4.95},
    "Este": {"almacen": "ALM-ESTE", "tarifa": 5.20},
}
# cómo acceder a un valor con doble clave
red["Norte"]["almacen"]

```

🔗 Pandas: DataFrame y Series

```

import pandas as pd

# cómo crear un DataFrame a partir de un diccionario
df = pd.DataFrame({
    "codigo": ["A1", "A2", "A3"],
    "ciudad": ["Madrid", "Sevilla", "Vigo"],
})

# cómo poner una columna como índice
df = df.set_index("codigo")

# cómo leer un csv
ventas = pd.read_csv("../data/ventas_retail.csv")
empleados = pd.read_csv("../data/empleados.csv", index_col="id_empleado")

# cómo ver un vistazo rápido de los datos
ventas.head(8)    # primeras 8 filas
ventas.tail(3)    # últimas 3 filas
ventas.shape      # (filas, columnas)
ventas.info()     # tipos de dato por columna
ventas.isna().sum() # cuenta nulos por columna

# cómo seleccionar UNA columna (esto devuelve una Serie, tipo columna especial)
ventas["region"]

# cómo seleccionar VARIAS columnas a la vez (esto devuelve un DataFrame)
ventas[["region", "categoria", "unidades"]]

# cómo hacer slicing de FILAS con corchetes (por posición)
ventas[10:16]

# cómo seleccionar por ETIQUETA (nombre de fila/columna) con .loc
empleados.loc["EMP-1042"]    # una fila entera
empleados.loc["EMP-1042", "departamento"]    # una celda concreta
empleados.loc[["EMP-1005", "EMP-1020"], ["departamento", "modalidad"]] # varias filas/columnas
empleados.loc[:, ["nombre", "bonus_pct"]]    # todas las filas, columnas concretas

# cómo seleccionar por POSICIÓN (números de fila/columna) con .iloc
empleados.iloc[0:3, [2, 5]]    # 3 primeras filas, columnas en posición 2 y 5

# cómo crear una columna calculada nueva (se aplica a TODA la columna, sin bucle)
ventas["importe_bruto"] = ventas["unidades"] * ventas["precio_unitario"]

# cómo agrupar y agregar datos (equivalente a un "GROUP BY" de SQL)

```

```

ventas.groupby("region")["importe_netto"].sum().sort_values(ascending=False)

# cómo agrupar por varias columnas y aplicar varias funciones a la vez
ventas.groupby(["categoria", "canal"]).agg({
    "importe_netto": "mean",
    "unidades": "sum",
})

# cómo obtener el top N de un ranking
ventas.groupby("producto")["importe_netto"].sum().nlargest(5)

# cómo rellenar valores nulos (NaN)
ventas["satisfaccion"] = ventas["satisfaccion"].fillna(ventas["satisfaccion"].median())
ventas["canal"] = ventas["canal"].fillna("Desconocido")

# cómo exportar un DataFrame a csv
resultado.to_csv("../outputs/facturacion_por_region.csv")

# cómo aplicar una función propia a cada valor de una columna (sin bucle for manual)
# OJO: se pasa el NOMBRE de la función, SIN paréntesis y SIN llamarla
sensores["riesgo"] = sensores["temperatura_c"].apply(nivel_riesgo)

# cómo contar cuántas veces aparece cada valor distinto de una columna
sensores["riesgo"].value_counts()

# cómo filtrar filas con una condición booleana
ventas[ventas["importe_netto"] > 500]

# cómo filtrar comprobando si el valor está DENTRO de una lista
ventas[ventas["region"].isin(["Norte", "Centro"])]

# cómo filtrar comprobando si un valor está en un rango
ventas[ventas["descuento_pct"].between(10, 20)]

# cómo combinar varias condiciones (¡cada una entre paréntesis!)
ventas[(ventas["categoria"] == "Electronica") & (ventas["unidades"] > 2) & (ventas["descuento_pct"] != 0)]

# cómo invertir un filtro (NO cumple la condición)
ventas[~(ventas["categoria"] == "Oficina")]

# cómo filtrar con sintaxis de "consulta" (más legible, como si fuera SQL)
ventas.query("canal == 'Online' and importe_netto > 500")

# cómo usar el accesor de texto ".str" para trabajar con columnas de texto
ventas["region"].str[:3].str.upper() # primeras 3 letras en mayúsculas

# cómo recorrer un DataFrame fila a fila (evitar si se puede, es lento)
for etiqueta, fila in ventas.head(5).iterrows():
    print(fila["region"], fila["importe_netto"])

```

◆ Operadores de comparación y lógicos

```

2 < 3          # True
"A" == "a"     # False (distingue mayúsculas/minúsculas)
3 != 3.0       # False (3 y 3.0 son iguales en valor)
"Z" > "M"      # True (compara alfabéticamente, letra a letra)

True + True + False # 2 -> en Python, True vale 1 y False vale 0

# operadores lógicos EN PYTHON NORMAL (con valores sueltos, no arrays)
True and False     # AND -> False
True or False      # OR -> True
not True           # NOT -> False

# con arrays de NumPy o columnas de pandas, "and"/"or" DAN ERROR (ValueError)
# hay que usar "&" y "|" con cada condición entre paréntesis:
(temp > 70) & (vib > 3)

```

◆ Condicionales if / elif / else


```
def nivel_riesgo(temperatura):
    if temperatura > 85:
        return "CRÍTICO"
    elif temperatura > 76:
        return "ALTO"
    elif temperatura > 70:
        return "MEDIO"
    else:
        return "BAJO"

# cómo hacer un if en una sola línea (expresión condicional / "operador ternario")
estado = "Caro" if precio > 100 else "Barato"
```

◆ Bucles `while` y `for`

```
# cómo hacer un bucle while (se repite MIENTRAS la condición sea True)
stock = 12
while stock < 60:
    stock = stock + 8
    print(stock)

# cómo hacer un bucle for sobre una lista
for region in ["Norte", "Sur", "Este"]:
    print(region.upper())

# cómo recorrer con índice y valor a la vez usando enumerate
for i, region in enumerate(["Norte", "Sur", "Este"], start=1):
    print(f"Región {i}: {region}")

# cómo recorrer una lista de listas desempaquetando cada fila
inventario = [{"ALM-NORTE", "Tablet", 34, 201.50}]
for almacen, producto, unidades, coste in inventario:
    print(almacen, producto, unidades * coste)

# cómo recorrer un diccionario
for clave, valor in {"a": 1, "b": 2}.items():
    print(clave, valor)

# cómo recorrer un array de NumPy
import numpy as np
for precio in np.array([10, 20, 30]):
    print(precio * 1.04)

# cómo construir una lista con un bucle de forma compacta (list comprehension)
cuadrados = [x**2 for x in range(5)] # [0, 1, 4, 9, 16]
valoraciones = [random.randint(1, 5) for _ in range(10)] # 10 valores aleatorios
```

◆ f-strings (cómo formatear texto con variables)

```
region = "Norte"
tarifa = 4.95
print(f"{region} -> {tarifa} €") # forma moderna y más usada, con f delante
print("{} -> {} €".format(region, tarifa)) # forma alternativa con .format()
```

Sección 3 — Listas

Ejercicio 1 — Catálogo de productos como lista

Enunciado: > Una tienda gestiona su catálogo en una lista plana que alterna nombre de producto y precio. 1) Imprime el número total de elementos y deduce cuántos productos hay. 2) Con slicing con paso, crea `nombres` y `precios`. 3) Obtén el precio del “Robot Aspirador” usando `.index()`, sin escribir el índice a mano. 4) Crea `ultimos_tres_nombres` con índices negativos. 5) Calcula el precio medio redondeado a dos decimales usando `sum()`, `len()` y `round()`.

Código en Python:

```
catalogo = [
    "Auriculares BT", 59.90,
    "Smartwatch S2", 149.00,
    "Tablet 10", 219.00,
    "Cargador USB-C", 18.50,
    "Cafetera Expres", 189.00,
    "Robot Aspirador", 279.00,
]

# 1. Número total de elementos
total_elementos = len(catalogo)
print("Elementos totales:", total_elementos)
print("Número de productos:", total_elementos // 2) # cada producto ocupa 2 posiciones

# 2. Slicing con paso: nombres en posiciones pares, precios en impares
nombres = catalogo[0:2:]
precios = catalogo[1:2:]
print("Nombres:", nombres)
print("Precios:", precios)

# 3. Precio del Robot Aspirador sin escribir el índice a mano
posicion_nombre = catalogo.index("Robot Aspirador")
precio_robot = catalogo[posicion_nombre + 1]
print("Precio Robot Aspirador:", precio_robot)

# 4. Últimos tres nombres con índices negativos
ultimos_tres_nombres = nombres[-3:]
print("Últimos tres nombres:", ultimos_tres_nombres)

# 5. Precio medio redondeado a 2 decimales
precio_medio = round(sum(precios) / len(precios), 2)
print("Precio medio:", precio_medio)
```

Explicación “Para Tontos”: - `catalogo` es una **lista plana** que mezcla texto y números alternados: posición 0 es un nombre, posición 1 su precio, posición 2 otro nombre, etc. - `len(catalogo)` cuenta **todos** los elementos (12 en total), por eso para saber cuántos productos hay dividimos entre 2 con `//` (división entera). - `catalogo[0:2:]` es slicing con **paso 2** empezando en la posición 0: coge las posiciones 0, 2, 4, 6... (todas las pares → los nombres). `catalogo[1:2:]` hace lo mismo pero empezando en 1 → posiciones impares → los precios. - `.index("Robot Aspirador")` busca la posición donde aparece ese texto **sin que tengas que contar a mano**; como sabemos que el precio va siempre justo después del nombre, sumamos 1 a esa posición para llegar al precio. - `nombres[-3:]` usa **índices negativos**: -1 es el último elemento, -2 el penúltimo, etc. `[-3:]` significa “desde el tercero por el final hasta el final”. - `sum(precios) / len(precios)` calcula la media a mano (suma total entre número de elementos), y `round(..., 2)` redondea a dos decimales.

Ejercicio 2 — Inventario multialmacén con listas anidadas

Enunciado: > El inventario de tres almacenes se representa como lista de listas `[codigo_almacen, producto, unidades, coste_unitario]`. 1) Imprime el producto de la cuarta fila con doble indexación. 2) Crea `valor_stock` con `[producto, unidades*coste]` redondeado, con un `for`. 3) Calcula el valor inmovilizado total. 4) Extrae con slicing la sublista del `ALM-SUR` y calcula su valor. 5) Explica por qué el slicing no es robusto si los almacenes dejan de estar contiguos.

Código en Python:

```

inventario = [
    ["ALM-NORTE", "Robot Aspirador", 34, 201.50],
    ["ALM-NORTE", "Monitor 27", 58, 142.00],
    ["ALM-SUR", "Silla Ergonomica", 12, 178.90],
    ["ALM-SUR", "Robot Aspirador", 7, 201.50],
    ["ALM-ESTE", "Cafetera Expres", 41, 131.20],
    ["ALM-ESTE", "Monitor 27", 25, 142.00],
]

# 1. Producto de la cuarta fila (índice 3) con doble indexación
producto_fila4 = inventario[3][1]
print("Producto fila 4:", producto_fila4)

# 2. Lista valor_stock: [producto, unidades * coste_unitario]
valor_stock = []
for fila in inventario:
    codigo_almacen, producto, unidades, coste = fila
    valor_stock.append([producto, round(unidades * coste, 2)])
print("Valor stock:", valor_stock)

# 3. Valor inmovilizado total
valor_total = round(sum(item[1] for item in valor_stock), 2)
print("Valor inmovilizado total:", valor_total)

# 4. Sublista del ALM-SUR (filas 2 y 3, contiguas) y su valor
filas_alm_sur = inventario[2:4]
valor_alm_sur = round(sum(fila[2] * fila[3] for fila in filas_alm_sur), 2)
print("Filas ALM-SUR:", filas_alm_sur)
print("Valor ALM-SUR:", valor_alm_sur)

```

Explicación “Para Tontos”: - `inventario[3][1]` es **doble indexación**: primero `inventario[3]` accede a la cuarta fila (posición 3, porque se empieza a contar en 0) que es `["ALM-SUR", "Robot Aspirador", 7, 201.50]`, y luego `[1]` coge el segundo elemento de esa sublista, el nombre del producto. - En el bucle `for fila in inventario`: se **desempaqueta** cada sublista en cuatro variables (`codigo_almacen, producto, unidades, coste = fila`) para no tener que escribir `fila[0]`, `fila[1]`, etc. - `valor_stock.append(...)` va **añadiendo** una lista `[producto, valor]` por cada vuelta del bucle; así se construye una lista nueva a partir de la original sin modificarla. - El valor inmovilizado total suma el segundo elemento (`item[1]`) de cada entrada de `valor_stock` usando una expresión generadora dentro de `sum()`. - `inventario[2:4]` funciona porque las dos filas del ALM-SUR están **una detrás de otra** (posiciones 2 y 3); el slicing `[2:4]` coge desde la posición 2 hasta la 3 (el 4 no se incluye). - **Punto 5 (respuesta)**: el slicing por posición asume que las filas de un mismo almacén están siempre seguidas. Si en algún momento se añaden más pedidos y las filas del ALM-SUR quedan intercaladas con las de otro almacén, el mismo slicing `[2:4]` dejaría de coger las filas correctas — por eso no es una estrategia robusta de filtrado; lo correcto sería filtrar **por condición** (comprobando el valor de `codigo_almacen`), no por posición fija.

Ejercicio 3 — Manipulación destructiva y copias de listas

Enunciado: > Trabaja el comportamiento de referencias frente a copias de listas: asignación directa (`=`), copias con `list()` y `[:]`, aplicar descuento con bucle, `.extend()` / `.insert()`, `del` / `.pop()`, y comprobación con `id()`.

Código en Python:

```

precios_originales = [59.90, 149.00, 219.00, 18.50, 189.00, 279.00]

# 1. Asignación directa -> crea una REFERENCIA, no una copia
precios_rebajados = precios_originales
precios_rebajados[0] = 49.90
print("Originales:", precios_originales) # también cambia: [49.90, ...]
print("Rebajados:", precios_rebajados)

# Restaurar el valor original
precios_originales[0] = 59.90

# 2. Copias independientes de verdad
copia_list = list(precios_originales)
copia_slicing = precios_originales[:]

copia_list[0] = 0.01
print("Original tras modificar copia_list:", precios_originales) # no cambia
copia_slicing[1] = 0.01
print("Original tras modificar copia_slicing:", precios_originales) # tampoco cambia

# 3. Aplicar rebaja del 10% a todos los elementos de una copia
copia_rebajada = list(precios_originales)
for i in range(len(copia_rebajada)):
    copia_rebajada[i] = round(copia_rebajada[i] * 0.9, 2)
print("Copia rebajada un 10%:", copia_rebajada)

# 4. Añadir productos: .extend() para varios, .insert() en una posición concreta
copia_rebajada.extend([99.00, 45.00])
copia_rebajada.insert(2, 15.00)
print("Tras extend e insert:", copia_rebajada)

# 5. Eliminar elementos: del por posición, pop() guardando el valor extraído
del copia_rebajada[4]
descartado = copia_rebajada.pop()
print("Tras eliminar:", copia_rebajada)
print("Descartado:", descartado)

# 6. Comprobación con id()
print(id(precios_originales) == id(copia_list)) # False
print(id(precios_originales) == id(copia_slicing)) # False

```

Explicación “Para Tontos”: - `precios_rebajados = precios_originales` **no crea una lista nueva**: ambas variables apuntan al **mismo objeto** en memoria. Por eso, al modificar `precios_rebajados[0]`, `precios_originales` también cambia — es el error de copias más típico y más caro de depurar. - `list(precios_originales)` y `precios_originales[:]` sí crean una **copia independiente** (un objeto nuevo en memoria con los mismos valores). A partir de ahí, modificar la copia ya no afecta a la original. - Para aplicar el descuento a todos los elementos se usa `range(len(copia_rebajada))` para generar los índices 0, 1, 2, ... y así poder **reasignar** cada posición (`copia_rebajada[i] = ...`); un `for` elemento in lista normal no serviría porque elemento sería solo una copia del valor, no una referencia a la posición. - `.extend([...])` añade **varios elementos al final** de golpe; `.insert(posicion, valor)` mete un único elemento en una posición concreta, desplazando el resto hacia la derecha. - `del copia_rebajada[4]` elimina el elemento en la posición 4 y no devuelve nada; `.pop()` (sin argumento) elimina y **devuelve** el último elemento, que aquí se guarda en `descartado`; `.pop(0)` haría lo mismo pero con el primero. - `id()` devuelve la **dirección de memoria** del objeto; si dos variables tienen el mismo `id()`, son literalmente el mismo objeto (como en el paso 1); si tienen `id()` distinto, son copias independientes (como en el paso 2).

Ejercicio 4 — Cola de pedidos pendientes

Enunciado: > Cola de pedidos ordenada por prioridad. 1) Invierte con slicing negativo. 2) Ordena con `sorted()` sin modificar la original. 3) Ordena en sitio con `.sort(reverse=True)`. 4) Atiende los dos primeros con `.pop(0)`. 5) Inserta un pedido urgente al principio. 6) Informe final con atendidos, pendientes y porcentaje.

Código en Python:

```

cola = ["PED-10021", "PED-10007", "PED-10044", "PED-10012", "PED-10033", "PED-10008"]
total_inicial = len(cola)

# 1. Invertir con slicing con paso negativo
cola_invertida = cola[::-1]
print("Cola invertida:", cola_invertida)

# 2. Ordenar sin modificar la original
cola_ordenada = sorted(cola)
print("Cola ordenada (nueva):", cola_ordenada)
print("Cola original intacta:", cola)

# 3. Ordenar en sitio, de mayor a menor
cola.sort(reverse=True)
print("Cola tras sort en sitio:", cola)

# 4. Atender los dos primeros pedidos
atendidos = []
atendidos.append(cola.pop(0))
atendidos.append(cola.pop(0))
print("Atendidos:", atendidos)
print("Cola tras atender:", cola)

# 5. Insertar un pedido urgente al principio
cola.insert(0, "PED-99999")
print("Cola con pedido urgente:", cola)

# 6. Informe final
pendientes = len(cola)
porcentaje_avance = round(len(atendidos) / total_inicial * 100, 1)
print(f"Pedidos atendidos: {len(atendidos)}")
print(f"Pedidos pendientes: {pendientes}")
print(f"Porcentaje de avance: {porcentaje_avance}%")

```

Explicación “Para Tontos”: - `cola[::-1]` recorre la lista con **paso -1**, es decir, de atrás hacia adelante — una forma muy usada de invertir una lista sin usar `.reverse()`. - `sorted(cola)` **devuelve una lista nueva** ordenada y deja `cola` tal cual estaba; en cambio `cola.sort(reverse=True)` ordena **la propia lista** (la modifica) y no devuelve nada útil (`None`), por eso no se hace `cola = cola.sort(...)`. - `.pop(0)` elimina y devuelve el **primer** elemento de la lista (a diferencia de `.pop()` sin argumento, que quita el último); se usa dos veces para atender los dos pedidos con mayor prioridad, guardando cada uno en `atendidos`. - `.insert(0, "PED-99999")` mete el pedido urgente **en la posición 0**, desplazando todo lo demás una posición hacia atrás. - El porcentaje de avance compara cuántos pedidos se han atendido (`len(atendidos)`) frente al total que había al principio (`total_inicial`), multiplicado por 100 y redondeado a un decimal.

Sección 4 — Funciones y Paquetes

Ejercicio 5 — Funciones propias para el cálculo de importes

Enunciado: > Define `calcular_importe(unidades, precio_unitario, descuento_pct=0)` y `resumen_pedido(...)` que devuelva una tupla (`importe_bruto`, `ahorro`, `importe_netto`). Llama a `resumen_pedido` con argumentos por palabra clave y desempaqueta el resultado. Documenta con docstring y usa `max()`, `min()`, `sorted()`, `len()` sobre los importes de 5 pedidos.

Código en Python:

```
def calcular_importe(unidades, precio_unitario, descuento_pct=0):
    """Devuelve el importe neto de una línea de pedido, redondeado a 2 decimales."""
    bruto = unidades * precio_unitario
    return round(bruto * (1 - descuento_pct / 100), 2)

def resumen_pedido(unidades, precio_unitario, descuento_pct=0):
    """Devuelve una tupla (importe_bruto, ahorro, importe_netto) de un pedido."""
    bruto = round(unidades * precio_unitario, 2)
    neto = calcular_importe(unidades, precio_unitario, descuento_pct)
    ahorro = round(bruto - neto, 2)
    return bruto, ahorro, neto

# Llamada con argumentos por palabra clave (keyword arguments)
importe_bruto, ahorro, importe_netto = resumen_pedido(
    unidades=3, precio_unitario=59.90, descuento_pct=10
)
print("Bruto:", importe_bruto, "| Ahorro:", ahorro, "| Neto:", importe_netto)

# Documentación
help(calcular_importe)

# max, min, sorted, len sobre los importes de varios pedidos
pedidos = [
    (3, 59.90, 10),
    (1, 279.00, 0),
    (5, 18.50, 20),
    (2, 149.00, 5),
    (4, 89.90, 15),
]

importes = [calcular_importe(u, p, d) for u, p, d in pedidos]
print("Importes:", importes)
print("Nº de pedidos:", len(importes))
print("Importe máximo:", max(importes))
print("Importe mínimo:", min(importes))
print("Importes ordenados:", sorted(importes))

print(calcular_importe(3, 59.90, 10)) # comprobación -> 161.73
```

Explicación “Para Tontos”: - `descuento_pct=0` es un **parámetro con valor por defecto**: si al llamar a la función no le pasas ese argumento, Python usa automáticamente `0`, así que la función también funciona con solo dos argumentos:

`calcular_importe(3, 59.90)`. - `resumen_pedido` **devuelve tres valores a la vez** separados por comas (`return bruto, ahorro, neto`); Python empaqueta automáticamente esos tres valores en una **tupla**. - Al llamar con

`resumen_pedido(unidades=3, precio_unitario=59.90, descuento_pct=10)` se usan **argumentos por palabra clave**: en vez de depender del orden de los parámetros, se nombra explícitamente cada uno — más legible y con menos posibilidad de error si hay muchos parámetros. - `importe_bruto, ahorro, importe_netto = resumen_pedido(...)` es **desempaquetado de tupla**: Python reparte automáticamente los tres valores devueltos en las tres variables, en el mismo orden en que se devolvieron. - El texto entre `"""..."""` justo debajo de `def` es el **docstring**; `help(calcular_importe)` lo muestra por pantalla junto con la firma de la función. - La lista `importes` se construye con una **list comprehension**:

`[calcular_importe(u, p, d) for u, p, d in pedidos]` recorre cada tupla de `pedidos`, la desempaqueta en `u`, `p`, `d` y aplica la función — es una forma compacta de escribir un bucle `for` que va añadiendo resultados a una lista. - `max()`, `min()` y `sorted()` funcionan directamente sobre listas de números: dan el mayor, el menor y una versión ordenada respectivamente, sin modificar la lista original.

Ejercicio 6 — Normalización de códigos SKU con métodos

Enunciado: > Limpia códigos SKU con formatos inconsistentes: `.strip()` + `.upper()`, cuenta apariciones con `.count()`, busca posición con `.index()`, separa con `.split("-")`, filtra los de país "ES" y sustituye con `.replace()`, y únelos con `.join()`.

Código en Python:

```
skus_crudos = [
    " elc-0012-es ", "HOG-0045-ES", "dep-0003-pt ",
    "ofi-0021-es", " ELC-0012-ES", "hog-0099-fr ",
]

# 1. Limpieza: quitar espacios y pasar a mayúsculas
skus_limpios = [sku.strip().upper() for sku in skus_crudos]
print("SKU limpios:", skus_limpios)

# 2. Contar apariciones de "ELC-0012-ES"
apariciones = skus_limpios.count("ELC-0012-ES")
print("Apariciones de ELC-0012-ES:", apariciones)

# 3. Posición de la primera aparición de "OFI-0021-ES"
posicion = skus_limpios.index("OFI-0021-ES")
print("Posición de OFI-0021-ES:", posicion)

# 4. Separar cada SKU en familia, número y país
partes_sku = [sku.split("-") for sku in skus_limpios]
print("SKU separados:", partes_sku)

# 5. Filtrar los de país "ES" y sustituir el guion por un punto
skus_es = [sku.replace("-", ".") for sku in skus_limpios if sku.endswith("-ES")]
print("SKU de España (con punto):", skus_es)

# 6. Unir en una única cadena separada por " | "
cadena_unica = " | ".join(skus_limpios)
print("Cadena única:", cadena_unica)
```

Explicación “Para Tontos”: - `sku.strip().upper()` se puede **encadenar**: primero `.strip()` quita los espacios sobrantes al principio y al final, y sobre el resultado se aplica `.upper()` para pasar todo a mayúsculas. Ninguno de los dos modifica el string original (son inmutables), así que hay que guardar el resultado en una lista nueva. - `.count("ELC-0012-ES")` recorre la lista y cuenta **cuántas veces** aparece exactamente ese valor. - `.index("OFI-0021-ES")` devuelve la **posición** de la primera vez que aparece ese valor en la lista; si no existiera, daría un error `ValueError`. - `.split("-")` trocea cada string usando el guion como separador, y devuelve una **lista** con las partes: "ELC-0012-ES" se convierte en ["ELC", "0012", "ES"]. - El filtro `if sku.endswith("-ES")` dentro de la list comprehension solo deja pasar los SKU que **terminan en “-ES”** (los de España); sobre esos, `.replace("-", ".")` cambia todos los guiones por puntos. - `.join()` es un método de **string** (no de lista): `" | ".join(lista)` une todos los elementos de la lista en un único texto, colocando `" | "` entre cada par de elementos — es lo contrario de `.split()`.

Ejercicio 7 — Paquetes, importaciones y módulo propio

Enunciado: > Importa `math` completo y calcula raíz cuadrada; importa `pi` y `ceil` selectivamente para calcular palés necesarios; importa `numpy/pandas` con alias e imprime versiones; usa `random` con semilla fija; crea `src/utilidades.py` con `calcular_importe` y `clasificar_ticket`, e impórtalo desde el notebook.

Código en Python:

```

# 1. Importación completa del paquete
import math
raiz = math.sqrt(2809)
print("Raíz cuadrada de 2809:", raiz)

# 2. Importación selectiva
from math import pi, ceil
palets_necesarios = ceil(1375 / 24)
print("Palés necesarios:", palets_necesarios)

# 3. numpy y pandas con alias
import numpy as np
import pandas as pd
print("NumPy:", np.__version__)
print("pandas:", pd.__version__)

# 4. random con semilla fija
import random
random.seed(42)
valoraciones = [random.randint(1, 5) for _ in range(10)]
print(valoraciones)

# 5. (respuesta en la explicación de abajo)

# 6. Contenido de src/utilidades.py
# -----
# """Funciones auxiliares de la Práctica 3."""
#
# def calcular_importe(unidades, precio_unitario, descuento_pct=0):
#     """Devuelve el importe neto de una línea de pedido."""
#     bruto = unidades * precio_unitario
#     return round(bruto * (1 - descuento_pct / 100), 2)
#
# def clasificar_ticket(importe):
#     """Clasifica un importe en 'Bajo', 'Medio' o 'Alto'."""
#     if importe < 100:
#         return "Bajo"
#     elif importe < 400:
#         return "Medio"
#     return "Alto"
# -----

# 7. Importar el módulo propio desde el notebook
import sys
sys.path.append("../src")

from utilidades import calcular_importe, clasificar_ticket

print(calcular_importe(4, 89.90, 15)) # 305.66
print(clasificar_ticket(305.66))      # "Medio"

```

Explicación “Para Tontos”: - `import math` trae **todo** el módulo `math`; para usar cualquiera de sus funciones hay que escribir el prefijo `math.` delante (`math.sqrt`, `math.ceil`...). - `from math import pi, ceil` trae **solo esas dos** funciones/constantas concretas, y ya se pueden usar directamente sin el prefijo `math.` — más cómodo si solo necesitas un par de cosas del módulo. - Para calcular los palés necesarios: `1375 / 24` da un número con decimales (no se puede enviar “medio palé”), así que se usa `ceil()` (redondeo **hacia arriba**) para asegurarse de que hay palés suficientes para todas las unidades. - `import numpy as np` y `import pandas as pd` son la forma **estándar** de importar estas dos librerías con un alias corto, porque se usan constantemente en el código y escribir `numpy.algo` todo el rato sería muy largo. - `random.seed(42)` **fija la semilla** del generador de números aleatorios: a partir de ahí, todas las llamadas a `random.randint(...)` seguirán siempre la misma secuencia “aleatoria” predecible. - **Punto 5 (respuesta):** fijar la semilla hace que el generador de números pseudoaleatorios siempre arranque desde el mismo punto interno, así que produce exactamente la misma secuencia de números cada vez que se ejecuta el código — esto permite que tú y el profesor obtengáis el mismo resultado al ejecutar la misma celda. - `sys.path.append("../src")` añade la carpeta `src/` (que está un nivel por encima de `notebooks/`) a la lista de sitios donde Python busca módulos para importar; sin esa línea, `from utilidades import ...` fallaría con un `ModuleNotFoundError` porque el notebook no “ve” por defecto esa carpeta.

Sección 5 — NumPy

Ejercicio 8 — Del cálculo con listas al cálculo vectorizado

Enunciado: > Calcula `unidades * precios` con listas de Python (y explica el resultado/error). Convierte a arrays NumPy, calcula `dtype/shape`, el array `importes` y su suma. Aplica un recargo del 4%. Crea un array mixto y explica el `dtype` resultante. Compara tiempos con `%timeit`.

Código en Python:

```
unidades = [3, 1, 5, 2, 4, 6, 2, 8, 3, 1]
precios = [59.90, 279.00, 18.50, 149.00, 89.90, 27.50, 219.00, 14.90, 64.00, 239.00]

# 1. Intento con listas normales
try:
    resultado = unidades * precios
except TypeError as error:
    print("Error:", error)
# unidades * precios da TypeError: no se pueden multiplicar dos listas entre sí

import numpy as np

# 2. Conversión a arrays NumPy
np_unidades = np.array(unidades)
np_precios = np.array(precios)
print("dtype unidades:", np_unidades.dtype, "| shape:", np_unidades.shape)
print("dtype precios:", np_precios.dtype, "| shape:", np_precios.shape)

# 3. Array de importes y suma total
importes = np_unidades * np_precios
print("Importes:", importes)
print("Suma total:", round(importes.sum(), 2))

# 4. Recargo logístico del 4% (operación vectorizada, sin bucle)
importes_con_recargo = importes * 1.04
print("Importes con recargo:", importes_con_recargo)

# 5. Array mixto
mixto = np.array([1, "dos", 3.0, True])
print("dtype del array mixto:", mixto.dtype)
print(mixto)

# 6. Comparación de tiempos (en una celda de JupyterLab, con magic command)
# %timeit [u * p for u, p in zip(unidades, precios)]
# %timeit np_unidades * np_precios

print("Productos con precio > 100:", (np_precios > 100).sum())
```

Explicación “Para Tontos”: - `unidades * precios` con **listas normales de Python** no multiplica elemento a elemento: Python interpreta el `*` entre una lista y otra como una operación no soportada entre dos listas, y lanza un `TypeError`. (Si fuera `lista * 3` con un número, sí funcionaría, pero repitiendo la lista 3 veces, no multiplicando sus valores). - Al convertir con `np.array(...)`, ambas listas pasan a ser **arrays de NumPy**, que sí soportan operaciones **vectorizadas**: `np_unidades * np_precios` multiplica automáticamente **posición a posición** (elemento 0 con elemento 0, elemento 1 con elemento 1...) sin necesidad de ningún bucle `for`. - `.dtype` indica el tipo de dato interno del array (`int64` para enteros, `float64` para decimales); `.shape` indica su forma — aquí, al ser arrays de una dimensión con 10 elementos, sería `(10,)`. - `importes * 1.04` aplica el recargo del 4% a **todos** los importes de golpe: NumPy “distribuye” automáticamente la operación con el número suelto (esto se llama *broadcasting*). - El array mixto `[1, "dos", 3.0, True]` mezcla enteros, texto, decimales y booleanos; como NumPy exige que **todos los elementos de un array tengan el mismo tipo**, convierte todo al tipo “más general” posible para que quepa: como hay texto, todo el array termina siendo de tipo string (`dtype='<U...'`), y los demás valores se convierten automáticamente a su representación en texto. - `%timeit` es un **comando mágico** de Jupyter (empieza por `%`) que ejecuta la celda muchas veces y mide el tiempo medio; normalmente la versión vectorizada con NumPy es notablemente más rápida que el bucle `for` equivalente en listas, sobre todo cuantos más elementos haya.

Ejercicio 9 — Subsetting con máscaras booleanas

Enunciado: > Crea la máscara `caros = np_precios > 100`, extrae los precios caros, cuenta cuántos hay con `.sum()`, aplica la misma máscara a `importes`, obtén posiciones con `np.where()`, y combina dos máscaras con `&`.

Código en Python:

```

import numpy as np

np_unidades = np.array([3, 1, 5, 2, 4, 6, 2, 8, 3, 1])
np_precios = np.array([59.90, 279.00, 18.50, 149.00, 89.90, 27.50, 219.00, 14.90, 64.00, 239.00])
importes = np_unidades * np_precios

# 1. Máscara booleana
caros = np_precios > 100
print("Máscara caros:", caros)

# 2. Precios que superan 100 €
print("Precios caros:", np_precios[caros])

# 3. Contar cuántos productos caros hay
num_caros = caros.sum()
print("Número de productos caros:", num_caros)

# 4. Aplicar la misma máscara a importes
importes_caros = importes[caros]
print("Importes de los productos caros:", importes_caros)
print("Suma de esos importes:", round(importes_caros.sum(), 2))

# 5. Posiciones que cumplen la condición
posiciones = np.where(caros)
print("Posiciones de productos caros:", posiciones)

# 6. Combinar dos máscaras con &
varias = np_unidades >= 2
seleccion = (np_precios > 100) & (np_unidades >= 2)
print("Importes que cumplen ambas condiciones:", importes[seleccion])

```

Explicación “Para Tontos”: - `np_precios > 100` no filtra nada todavía: crea un **array de la misma longitud** que `np_precios`, pero lleno de `True/False` según si cada posición cumple la condición o no — eso es una **máscara booleana**. - `np_precios[caros]` usa esa máscara como “índice”: NumPy solo se queda con los elementos donde la máscara vale `True`, descartando el resto. - `caros.sum()` funciona porque en Python (y en NumPy) `True` se comporta como `1` y `False` como `0`; al sumar todos los `True/False` del array, el resultado es directamente **el número de elementos que cumplen la condición**. - Como `caros` tiene la misma longitud y el mismo orden que `np_precios`, se puede reutilizar exactamente la misma máscara sobre `importes` (que también tiene esa longitud) para quedarnos con los importes de esos mismos productos caros. - `np.where(caros)` devuelve las **posiciones (índices)** donde la condición es `True`, en vez de los valores en sí — útil cuando necesitas saber “en qué fila” está algo, no solo su valor. - Para combinar condiciones con arrays de NumPy **no se usa `and/or`** (eso daría error), se usa `&` (Y) y `|` (O), y cada condición individual va **entre paréntesis** porque `&` tiene más prioridad que `>`, y sin paréntesis Python intentaría evaluar mal la expresión.

Ejercicio 10 — Matriz de ventas por trimestre y región

Enunciado: > Con una matriz `ventas_2d` (filas = trimestres, columnas = regiones): analiza `shape/ndim/size`, extrae filas/columnas, indexación 2D, slicing de submatriz, totales por eje, previsión de crecimiento apilada con `vstack`, y región líder con `argmax`.

Código en Python:

```

import numpy as np

# Filas: T1, T2, T3 | Columnas: Norte, Sur, Este, Oeste, Centro
ventas_2d = np.array([
    [15200.50, 11800.25, 12950.00, 9870.40, 13400.10],
    [16840.75, 12310.60, 13480.90, 11020.35, 14100.80],
    [12535.72, 10663.06, 11221.59, 12673.08, 12334.38],
])

regiones = ["Norte", "Sur", "Este", "Oeste", "Centro"]

# 1. shape, ndim y size
print("shape:", ventas_2d.shape) # (3 trimestres, 5 regiones)
print("ndim:", ventas_2d.ndim)   # 2 dimensiones (filas y columnas)
print("size:", ventas_2d.size)   # 15 valores de venta en total

# 2. Fila del T2 y columna de Centro
fila_t2 = ventas_2d[1, :]
columna_centro = ventas_2d[:, 4]
print("T2:", fila_t2)
print("Centro:", columna_centro)

# 3. Valor de T3 en la región Este (indexación 2D)
valor_t3_este = ventas_2d[2, 2]
print("T3-Este:", valor_t3_este)

# 4. Submatriz de T1 y T2 para las 3 primeras regiones
submatriz = ventas_2d[0:2, 0:3]
print("Submatriz T1-T2 / 3 primeras regiones:\n", submatriz)

# 5. Totales por trimestre y por región
total_por_trimestre = ventas_2d.sum(axis=1)
total_por_region = ventas_2d.sum(axis=0)
print("Total por trimestre:", total_por_trimestre)
print("Total por región:", total_por_region)

# 6. Previsión de crecimiento del 6% sobre T3, añadida como cuarta fila
prevision_t4 = ventas_2d[2, :] * 1.06
ventas_con_prevision = np.vstack([ventas_2d, prevision_t4])
print("Matriz con previsión T4:\n", ventas_con_prevision)

# 7. Región con mayor facturación acumulada
indice_lider = np.argmax(total_por_region)
print("Región líder:", regiones[indice_lider])

```

Explicación “Para Tontos”: - `shape` da la **forma** de la matriz como (filas, columnas): aquí (3, 5) porque hay 3 trimestres y 5 regiones. `ndim` es el **número de dimensiones**: 2, porque es una tabla (filas y columnas), no un simple array de una fila. `size` es el **total de valores** almacenados: $3 \times 5 = 15$. - `ventas_2d[1, :]` significa “fila en posición 1 (el T2), **todas** las columnas”; `ventas_2d[:, 4]` significa “**todas** las filas, columna en posición 4 (Centro)”. El `:` solo indica “coge todo en ese eje”. - `ventas_2d[2, 2]` es **indexación 2D** directa: fila 2 (T3), columna 2 (Este) — se accede a un único valor con las dos coordenadas separadas por coma, sin necesidad de encadenar corchetes como en las listas normales. - `ventas_2d[0:2, 0:3]` es slicing en dos dimensiones a la vez: filas de la 0 a la 1 (T1 y T2) y columnas de la 0 a la 2 (Norte, Sur, Este) — el resultado es una **submatriz** más pequeña. - `axis=1` suma **a lo largo de las columnas**, es decir, recorre cada fila y suma sus valores → da el total **por trimestre**. `axis=0` suma **a lo largo de las filas**, recorriendo cada columna → da el total **por región**. (Truco para no confundirse: `axis=0` colapsa las filas, `axis=1` colapsa las columnas). - `np.vstack([ventas_2d, prevision_t4])` **apila verticalmente** un array nuevo debajo de la matriz existente, como añadir una fila nueva a una tabla. - `np.argmax(total_por_region)` no da el valor máximo, sino la **posición** donde está ese máximo; por eso se usa esa posición para buscar el nombre correspondiente en la lista `regiones`.

Ejercicio 11 — Estadística descriptiva sobre telemetría simulada

Enunciado: > Genera con `rng.normal(...)` dos arrays de 500 elementos (temperatura y vibración), calcula media/mediana/desviación/min/max, percentiles con `np.percentile()`, apila con `column_stack`, calcula correlación con `corrcoef`, define un umbral de alerta, y explica la importancia de fijar la semilla.

Código en Python:

```

import numpy as np

rng = np.random.default_rng(2024)
temperatura = rng.normal(68, 6.5, 500)
vibracion = np.abs(rng.normal(2.4, 0.8, 500))

# 2. Estadísticos de temperatura
media_temp = temperatura.mean()
mediana_temp = np.median(temperatura)
desv_temp = temperatura.std()
min_temp = temperatura.min()
max_temp = temperatura.max()

print(f"Media: {media_temp:.2f} | Mediana: {mediana_temp:.2f}")
print(f"Desviación típica: {desv_temp:.2f}")
print(f"Mínimo: {min_temp:.2f} | Máximo: {max_temp:.2f}")

# 3. Percentiles de vibración
percentiles = np.percentile(vibracion, [25, 50, 75, 95])
print("Percentiles 25/50/75/95 de vibración:", percentiles)

# 4. Apilar en matriz (500, 2)
datos = np.column_stack([temperatura, vibracion])
print("Shape de la matriz apilada:", datos.shape)

# 5. Matriz de correlación
correlacion = np.corrcoef(temperatura, vibracion)
print("Matriz de correlación:\n", correlacion)

# 6. Umbral de alerta y conteo de lecturas que lo superan
umbral = media_temp + 2 * desv_temp
lecturas_altas = (temperatura > umbral).sum()
porcentaje = round(lecturas_altas / len(temperatura) * 100, 2)
print(f"Umbral: {umbral:.2f} | Lecturas que lo superan: {lecturas_altas} ({porcentaje}%)")

```

Explicación “Para Tontos”: - `rng = np.random.default_rng(2024)` crea un **generador de números aleatorios** con una semilla fija (2024); con esa misma semilla, `rng.normal(...)` producirá siempre los mismos 500 valores “aleatorios”, tanto en tu ordenador como en el del profesor. - `rng.normal(68, 6.5, 500)` genera 500 valores siguiendo una **distribución normal** (forma de campana) centrada en 68 (la media) con una dispersión típica de 6.5 (la desviación). - **Media vs mediana:** la media es el promedio de todos los valores; la mediana es el valor central si se ordenaran todos de menor a mayor. Si ambas son muy parecidas, los datos están bastante equilibrados (sin valores extremos que distorsionen la media); si difieren mucho, suele haber valores atípicos tirando de la media hacia un lado. - `np.percentile(vibracion, [25, 50, 75, 95])` calcula de golpe varios **percentiles**: por ejemplo, el percentil 95 es el valor por debajo del cual está el 95% de los datos — muy útil para detectar el rango de valores “normales” frente a los extremos. - `np.column_stack([temperatura, vibracion])` junta dos arrays de una dimensión (500 elementos cada uno) en una única matriz de forma (500, 2), donde cada fila representa una lectura con sus dos variables juntas. - `np.corrcoef(a, b)` devuelve una **matriz de correlación** 2x2; el valor fuera de la diagonal (por ejemplo, la posición [0,1]) indica cuán relacionadas están linealmente las dos variables, entre -1 (relación inversa perfecta) y 1 (relación directa perfecta); un valor cercano a 0 indica que, en estos datos simulados de forma independiente, **no hay relación lineal apreciable** entre temperatura y vibración (tiene sentido, porque se generaron con `rng.normal` de forma independiente). - El umbral `media + 2 * desviación` es un criterio estadístico habitual para marcar valores “atípicamente altos” (aproximadamente el 2-3% superior de una distribución normal); `(temperatura > umbral).sum()` cuenta cuántas lecturas superan ese umbral, reutilizando el mismo truco de sumar una máscara booleana del ejercicio 9. - Fijar la semilla es imprescindible para la **reproducibilidad**: sin ella, cada vez que se ejecute el notebook (tú, el profesor, o tú mismo en otro momento) se generarían datos aleatorios distintos, y sería imposible comparar resultados o verificar que el análisis es correcto.

Sección 6 — Diccionarios y Pandas

Ejercicio 12 — Diccionarios simples y anidados

Enunciado: > Crea `tarifas_envio`, accede, añade/actualiza claves, elimina con `del`, recorre con `.items()`, y construye un diccionario anidado `red_logistica` con acceso por doble clave.

Código en Python:

```
# 1. Diccionario de tarifas
tarifas_envio = {
    "Norte": 4.95,
    "Sur": 5.50,
    "Este": 5.20,
    "Oeste": 6.10,
    "Centro": 3.90,
}

# 2. Acceso a la tarifa de Este
print("Tarifa Este:", tarifas_envio["Este"])

# 3. Añadir Insular y actualizar Oeste
tarifas_envio["Insular"] = 9.80
tarifas_envio["Oeste"] = 5.95

# 4. Eliminar Sur y comprobar
del tarifas_envio["Sur"]
print("¿Existe Sur?", "Sur" in tarifas_envio)

# 5. Recorrer con .items()
for region, tarifa in tarifas_envio.items():
    print(f"{region} -> {tarifa} €")

# 6. Diccionario anidado
red_logistica = {
    "Norte": {"almacen": "ALM-NORTE", "tarifa": 4.95, "entrega_h": 24},
    "Este": {"almacen": "ALM-ESTE", "tarifa": 5.20, "entrega_h": 48},
    "Centro": {"almacen": "ALM-CENTRO", "tarifa": 3.90, "entrega_h": 24},
}

# 7. Tiempo de entrega del almacén del Este (doble clave)
print("Entrega Este:", red_logistica["Este"]["entrega_h"], "horas")

# 8. Añadir Oeste completo y recorrer toda la estructura
red_logistica["Oeste"] = {"almacen": "ALM-OESTE", "tarifa": 5.95, "entrega_h": 48}

for region, datos in red_logistica.items():
    print(f"{region}: almacén {datos['almacen']}, tarifa {datos['tarifa']} €, entrega {datos['entrega_h']}h")
```

Explicación “Para Tontos”: - Un diccionario se define con llaves `{}` y pares `clave: valor` separados por comas; `tarifas_envio["Este"]` accede al valor asociado a la clave "Este" directamente, sin necesidad de recorrer nada. - Para **añadir o actualizar** una clave se usa la misma sintaxis (`diccionario[clave] = valor`): si la clave ya existe, se sobrescribe su valor; si no existe, se crea una nueva entrada — Python decide automáticamente cuál de las dos cosas hacer. - `del tarifas_envio["Sur"]` elimina esa entrada por completo; el operador `in` (`"Sur" in tarifas_envio`) comprueba si una clave **sigue existiendo** en el diccionario, devolviendo `True/False`. - `.items()` devuelve **pares (clave, valor)** que se pueden desempaquetar directamente en el `for`; si solo escribieras `for region in tarifas_envio`: obtendrías únicamente las claves, sin sus valores. - Un **diccionario anidado** es un diccionario cuyos valores son, a su vez, otros diccionarios; para llegar a un dato concreto hay que encadenar dos claves: `red_logistica["Este"]["entrega_h"]` primero entra en la entrada "Este" (que es un diccionario) y luego coge de ahí la clave "entrega_h". - **Punto 9 (respuesta):** conviene un diccionario en vez de una lista cuando el **acceso a los datos tiene sentido por nombre** en vez de por posición — por ejemplo, `tarifas["Norte"]` es mucho más claro y menos propenso a errores que recordar que la tarifa de Norte está en la posición 3 de una lista; además, con un diccionario no importa el orden en que se añadieron las claves para poder acceder a ellas.

Ejercicio 13 — De diccionario a DataFrame y lectura de CSV

Enunciado: > Convierte un diccionario en DataFrame, pon una columna como índice con `.set_index()`, carga los tres CSV, inspecciona con `head()`/`tail()`/`shape`/`.info()`, y cuenta nulos con `.isna().sum()`.

Código en Python:

```

import pandas as pd

# 1. Diccionario a DataFrame
almacenes_dict = {
    "codigo": ["ALM-NORTE", "ALM-SUR", "ALM-ESTE", "ALM-OESTE", "ALM-CENTRO"],
    "ciudad": ["Bilbao", "Sevilla", "Valencia", "Vigo", "Madrid"],
    "m2": [4200, 3100, 3800, 2600, 6500],
    "operarios": [48, 31, 39, 22, 74],
}
almacenes = pd.DataFrame(almacenes_dict)

# 2. Ver el DataFrame con índice numérico automático
print(almacenes)

# 3. Convertir "codigo" en índice
almacenes = almacenes.set_index("codigo")
print(almacenes)

# 4. Carga de los tres csv
ventas = pd.read_csv("../data/ventas_retail.csv")
empleados = pd.read_csv("../data/empleados.csv", index_col="id_empleado")
sensores = pd.read_csv("../data/sensores_planta.csv")

# 5. head, tail y shape de ventas
print(ventas.head(8))
print(ventas.tail(3))
print("Shape ventas:", ventas.shape)

# 6. Info de tipos de columna
ventas.info()

# 7. Valores nulos por columna
print(ventas.isna().sum())

# 8. Shape de los tres DataFrames
print("Ventas:", ventas.shape)
print("Empleados:", empleados.shape)
print("Sensores:", sensores.shape)

```

Explicación “Para Tontos”: - `pd.DataFrame(diccionario)` convierte un diccionario en una tabla: **cada clave se convierte en una columna**, y las listas asociadas a cada clave se convierten en los valores de esa columna, fila por fila. - Por defecto, pandas asigna un **índice numérico automático** (0, 1, 2...) a cada fila; con `.set_index("codigo")` se le dice a pandas que use en su lugar la columna `"codigo"` como identificador de cada fila, algo típico cuando esa columna ya es un identificador único. - `pd.read_csv("ruta")` lee un fichero CSV y lo convierte directamente en un DataFrame; el parámetro `index_col="id_empleado"` hace lo mismo que `.set_index()` pero **durante la propia lectura**, en un solo paso. - `.head(8)` muestra las **primeras 8 filas** (útil para ver un vistazo rápido de los datos), `.tail(3)` muestra las **últimas 3**, y `.shape` da una tupla (filas, columnas) con el tamaño total de la tabla. - `.info()` imprime, columna por columna, cuántos valores no nulos tiene y **de qué tipo de dato es** (int64, float64, object para texto) — muy útil para detectar rápidamente qué columnas son numéricas y cuáles de texto. - `.isna()` devuelve una tabla del mismo tamaño llena de True/False según si cada celda está vacía (nula) o no; `.sum()` sobre eso **suma los True por columna** (igual que con las máscaras de NumPy), dando el número de nulos de cada columna — así se detecta que los nulos se concentran en `canal` y `satisfaccion`.

Ejercicio 14 — Selección con corchetes, `loc` e `iloc`

Enunciado: > Sobre `empleados` (con `id_empleado` como índice): selecciona una columna como Serie y como DataFrame, varias columnas a la vez, slicing de filas por posición, filas/celdas con `.loc`, filas/columnas con `.iloc`, y explica la diferencia entre corchetes, `loc` e `iloc`.

Código en Python:

```

import pandas as pd
empleados = pd.read_csv("../data/empleados.csv", index_col="id_empleado")

# 1. Una columna como Serie y como DataFrame
salario_serie = empleados["salario_base"]
salario_df = empleados[["salario_base"]]
print(type(salario_serie))    # <class 'pandas.core.series.Series'>
print(type(salario_df))      # <class 'pandas.core.frame.DataFrame'>

# 2. Varias columnas a la vez
subconjunto = empleados[["departamento", "ciudad", "salario_base"]]
print(subconjunto.head())

# 3. Filas de la 10 a la 15 (por posición) con slicing de corchetes
print(empleados[10:16])

# 4. Fila completa de un empleado concreto con loc
print(empleados.loc["EMP-1042"])

# 5. Departamento y modalidad de varios empleados con loc
print(empleados.loc[["EMP-1005", "EMP-1020", "EMP-1099"], ["departamento", "modalidad"]])

# 6. Tres primeras filas y columnas en posición 2 y 5 con iloc
print(empleados.iloc[0:3, [2, 5]])

# 7. Todas las filas, solo columnas nombre y bonus_pct con loc
print(empleados.loc[:, ["nombre", "bonus_pct"]])

```

Explicación “Para Tontos”: - `empleados["salario_base"]` (con corchete simple y un único nombre de columna) devuelve una **Serie**, que es como una única columna con su índice; `empleados[["salario_base"]]` (con doble corchete, una **lista** con un solo nombre) devuelve en cambio un **DataFrame** de una sola columna. La diferencia visual es sutil pero el tipo de objeto resultante es distinto. - `empleados[["departamento", "ciudad", "salario_base"]]` pasa una **lista de nombres de columna** entre corchetes para seleccionar varias a la vez. - `empleados[10:16]` con corchetes normales sobre un DataFrame hace slicing **por posición de fila** (como en una lista), igual que en listas de Python. - `.loc` selecciona **por etiqueta** (nombre de índice o de columna): `.loc["EMP-1042"]` busca la fila cuyo índice es literalmente ese texto; `.loc[[lista_filas], [lista_columnas]]` permite elegir varias filas y columnas concretas por su nombre a la vez. - `.iloc` selecciona **por posición numérica**, igual que en una lista o un array: `.iloc[0:3, [2, 5]]` coge las tres primeras filas (posiciones 0, 1, 2) y las columnas que están en la posición 2 y la posición 5, sin importar cómo se llamen. - `empleados.loc[:, ["nombre", "bonus_pct"]]` usa `:` para decir “todas las filas” y una lista de columnas para seleccionar solo esas dos.

Forma	Trabaja por...	Ejemplo
Corchetes <code>df[...]</code>	Columna(s) por nombre, o filas por posición con slicing	<code>df["col"]</code> , <code>df[10:16]</code>
<code>.loc[...]</code>	Etiqueta (nombre de índice/columna)	<code>df.loc["EMP-1042", "ciudad"]</code>
<code>.iloc[...]</code>	Posición numérica (como en listas)	<code>df.iloc[0:3, [2, 5]]</code>

Ejercicio 15 — Columnas calculadas, agregación y exportación

Enunciado: > Sobre `ventas`: crea `importe_bruto`, `importe_netto`, `ticket_medio`; agrupa por región (suma ordenada); agrupa por categoría y canal con `.agg()`; obtén el top 5 de productos con `.groupby()` + `.nlargest()`; rellena nulos de `satisfacción` y `canal`; exporta el resultado agrupado por región.

Código en Python:

```

import pandas as pd
ventas = pd.read_csv("../data/ventas_retail.csv")

# 1. Importe bruto
ventas["importe_bruto"] = ventas["unidades"] * ventas["precio_unitario"]

# 2. Importe neto aplicando descuento
ventas["importe_netto"] = round(
    ventas["importe_bruto"] * (1 - ventas["descuento_pct"] / 100), 2
)

# 3. Ticket medio
ventas["ticket_medio"] = round(ventas["importe_netto"] / ventas["unidades"], 2)

# 4. Suma de importe_netto por región, ordenada de mayor a menor
por_region = ventas.groupby("region")["importe_netto"].sum().sort_values(ascending=False)
print(por_region)

# 5. Agrupar por categoría y canal, con varias agregaciones
por_categoria_canal = ventas.groupby(["categoria", "canal"]).agg({
    "importe_netto": "mean",
    "unidades": "sum",
})
print(por_categoria_canal)

# 6. Top 5 de productos por facturación neta
top5_productos = ventas.groupby("producto")["importe_netto"].sum().nlargest(5)
print(top5_productos)

# 7. Rellenar nulos
ventas["satisfaccion"] = ventas["satisfaccion"].fillna(ventas["satisfaccion"].median())
ventas["canal"] = ventas["canal"].fillna("Desconocido")

# 8. Exportar el resultado agrupado por región
por_region.to_csv("../outputs/facturacion_por_region.csv")

print("Facturación neta total:", round(ventas["importe_netto"].sum(), 2))

```

Explicación “Para Tontos”: - `ventas["unidades"] * ventas["precio_unitario"]` es una operación **vectorizada de pandas**: multiplica columna contra columna, fila por fila, sin necesidad de ningún bucle — el resultado se guarda directamente como una **columna nueva** al asignarlo a `ventas["importe_bruto"]`. - `ventas["importe_bruto"] * (1 - ventas["descuento_pct"] / 100)` aplica la fórmula del descuento a toda la columna de golpe: si el descuento es 10 (%), $1 - 10/100 = 0.9$, así que se multiplica el bruto por 0.9 para obtener el neto. - `.groupby("region")` agrupa todas las filas que comparten el mismo valor de "region"; `["importe_netto"].sum()` suma esa columna **dentro de cada grupo**; `.sort_values(ascending=False)` ordena el resultado de mayor a menor. - `.agg({"columna": "función"})` permite aplicar **una función distinta a cada columna** dentro de un mismo `groupby`: aquí se pide la **media** de `importe_netto` y la **suma** de `unidades`, agrupando simultáneamente por dos columnas (`categoria` y `canal`). - `.nlargest(5)` sobre el resultado de un `groupby` + `sum()` da directamente los **5 valores más altos**, ya ordenados — combina agrupar, sumar y quedarte con el top todo en una línea. - `.fillna(valor)` sustituye los huecos vacíos (NaN) de una columna por el valor que le indiques; para `satisfaccion` se usa la **mediana** de la propia columna (una forma habitual de rellenar nulos numéricos sin distorsionar demasiado la distribución), y para `canal` (texto) se usa directamente el texto "Desconocido". - `.to_csv("ruta")` guarda cualquier DataFrame o Series en un fichero CSV en la ruta indicada — aquí se guarda el resumen ya agrupado por región, no la tabla `ventas` completa.

Sección 7 — Lógica, Control de Flujo y Filtrado

Ejercicio 16 — Operadores de comparación y booleanos

Enunciado: > Evalúa varias expresiones booleanas, construye condiciones combinadas con `and/or/not`, reproduce el `ValueError` al usar `and/or` con arrays de NumPy, y resuélvelo con `np.logical_and()` y con `&`.

Código en Python:

```
import numpy as np

# 1. Expresiones básicas
print(2 < 3)                # True: 2 es menor que 3
print("Norte" == "norte")   # False: distingue mayúsculas de minúsculas
print(True + True + False)  # 2: True vale 1, False vale 0 -> 1+1+0
print(3 != 3.0)             # False: 3 y 3.0 representan el mismo valor numérico
print("Zapatilla" > "Mochila") # True: compara alfabéticamente letra a letra ('Z' > 'M')

# 2. Condición: más de 4 unidades Y descuento >= 10
unidades = 6
precio = 149.0
descuento = 15
cumple_condicion_2 = (unidades > 4) and (descuento >= 10)
print("Condición 2:", cumple_condicion_2) # True

# 3. Condición: precio < 50 O precio > 200
cumple_condicion_3 = (precio < 50) or (precio > 200)
print("Condición 3:", cumple_condicion_3) # False (149 no es ni tan bajo ni tan alto)

# 4. Negación de la condición 2
print("Negación de la condición 2:", not cumple_condicion_2) # False

# 5. Arrays para probar operadores booleanos con NumPy
temp = np.array([64.2, 88.1, 71.0, 92.5, 69.8])
vib = np.array([1.9, 3.4, 2.1, 4.0, 2.2])

# 6. Esto da ERROR a propósito
try:
    resultado = (temp > 70) and (vib > 3)
except ValueError as error:
    print("Error:", error)

# 7. Formas correctas de combinar condiciones con arrays
print(np.logical_and(temp > 70, vib > 3))
print((temp > 70) & (vib > 3))
```

Explicación “Para Tontos”: - `2 < 3` compara dos números → `True`. - `"Norte" == "norte"` compara dos strings **carácter a carácter, distinguiendo mayúsculas de minúsculas** → como “N” no es igual a “n”, el resultado es `False`. - `True + True + False` funciona porque en Python los booleanos son, por debajo, **números enteros** (`True = 1`, `False = 0`); sumarlos da un resultado numérico normal: `1 + 1 + 0 = 2`. - `3 != 3.0` compara un `int` con un `float`; Python los considera **iguales en valor** (aunque sean de tipo distinto), así que `!=` (distinto de) da `False`. - `"Zapatilla" > "Mochila"` compara strings usando el **orden alfabético** de sus caracteres (técnicamente, el código Unicode de cada letra); como “Z” va después de “M” en el alfabeto, el resultado es `True`. - `and` exige que **ambas condiciones sean verdaderas**; `or` con que **una sola** lo sea; `not` invierte el resultado. Estos operadores funcionan bien con valores sueltos (`True/False`), pero fallan con arrays. - El error del punto 6: `(temp > 70) and (vib > 3)` intenta usar `and` sobre dos **arrays enteros** (con varios `True/False` dentro cada uno), y Python no sabe cómo convertir un array completo en un único `True/False` para decidir el resultado de `and` — por eso lanza `ValueError: The truth value of an array with more than one element is ambiguous`. - La solución es usar `np.logical_and(a, b)` o el operador `&` (con cada condición entre paréntesis): estos sí operan **elemento a elemento**, devolviendo un array de `True/False` en vez de intentar reducirlo a un único valor.

Ejercicio 17 — `if / elif / else` aplicado a mantenimiento predictivo

Enunciado: > Escribe `nivel_riesgo(temperatura)` con niveles CRÍTICO/ALTO/MEDIO/BAJO, pruébala con 4 valores, aplícala a la columna de temperatura de `sensores` con `.apply()`, cuenta niveles con `.value_counts()`, filtra los críticos y averigua qué máquina acumula más.

Código en Python:

```

import pandas as pd

def nivel_riesgo(temperatura):
    """Clasifica una temperatura en un nivel de riesgo de mantenimiento."""
    if temperatura > 85:
        return "CRÍTICO"
    elif temperatura > 76:
        return "ALTO"
    elif temperatura > 70:
        return "MEDIO"
    else:
        return "BAJO"

# 2. Prueba con cuatro valores, uno por rama
print(nivel_riesgo(92.5)) # CRÍTICO
print(nivel_riesgo(80.0)) # ALTO
print(nivel_riesgo(72.3)) # MEDIO
print(nivel_riesgo(64.1)) # BAJO

# 3. Aplicar la función a la columna de temperatura de sensores
sensores = pd.read_csv("../data/sensores_planta.csv")
sensores["riesgo"] = sensores["temperatura_c"].apply(nivel_riesgo)
print(sensores[["id_maquina", "temperatura_c", "riesgo"]].head())

# 4. Contar lecturas por nivel de riesgo
print(sensores["riesgo"].value_counts())

# 5. Máquina con más lecturas en riesgo CRÍTICO
criticos = sensores[sensores["riesgo"] == "CRÍTICO"]
print(criticos["id_maquina"].value_counts())

# Comprobación del enunciado
print("Lecturas ALARMA:", (sensores["estado"] == "ALARMA").sum())
print("Lecturas AVISO:", (sensores["estado"] == "AVISO").sum())

```

Explicación “Para Tontos”: - El `if / elif / else` se evalúa **de arriba a abajo**, y se queda con la **primera** condición que sea verdadera: por eso el orden importa — primero se pregunta por el caso más extremo (`> 85`) y se va bajando; si pusieras primero `> 70`, todas las temperaturas altas también entrarían ahí y nunca llegarían a comprobar `> 76` o `> 85`. - `sensores["temperatura_c"].apply(nivel_riesgo)` aplica la función `nivel_riesgo` a **cada valor** de la columna `temperatura_c`, uno por uno, y devuelve una nueva columna con los resultados; es el equivalente en pandas a recorrer la columna con un bucle `for`, pero mucho más rápido y compacto. **Importante:** se pasa `nivel_riesgo` **sin paréntesis** — con paréntesis (`nivel_riesgo()`) Python intentaría **ejecutar** la función inmediatamente en vez de dársela a `.apply()` para que la use ella misma sobre cada valor. - `.value_counts()` cuenta cuántas veces aparece cada valor distinto en una columna, y lo devuelve ordenado de más frecuente a menos frecuente — muy útil para ver de un vistazo cuántas lecturas hay de cada nivel de riesgo. - `sensores[sensores["riesgo"] == "CRÍTICO"]` es un **filtro booleano**: `sensores["riesgo"] == "CRÍTICO"` genera una columna de `True/False`, y usarla dentro de los corchetes de `sensores[...]` se queda solo con las filas donde es `True`. - Sobre ese subconjunto filtrado, `["id_maquina"].value_counts()` cuenta cuántas lecturas críticas tiene cada máquina, y el primero de la lista (la más frecuente) es la que acumula más avisos críticos. - **Punto 6 (recomendación de negocio, ejemplo):** la máquina que más lecturas críticas acumula debería priorizarse para una revisión de mantenimiento preventivo antes de que se produzca una parada no planificada, ya que temperaturas por encima de 85 °C de forma repetida suelen ser un indicador temprano de fallo mecánico inminente.

Ejercicio 18 — Filtrado avanzado de DataFrames

Enunciado: > Sobre `ventas` y `empleados`: filtra por canal e importe, con `.isin()`, con `.between()`, combinando tres condiciones, invirtiendo con `~`, comparando con `.query()`, y filtra/exporta empleados de Ingeniería en remoto con antigüedad.

Código en Python:

```

import pandas as pd
ventas = pd.read_csv("../data/ventas_retail.csv")
ventas["importe_bruto"] = ventas["unidades"] * ventas["precio_unitario"]
ventas["importe_netto"] = round(ventas["importe_bruto"] * (1 - ventas["descuento_pct"] / 100), 2)
empleados = pd.read_csv("../data/empleados.csv", index_col="id_empleado")

# 1. Pedidos Online con importe_netto > 500
online_altos = ventas[(ventas["canal"] == "Online") & (ventas["importe_netto"] > 500)]
print(online_altos.shape)

# 2. Regiones Norte o Centro con isin
norte_centro = ventas[ventas["region"].isin(["Norte", "Centro"])]
print(norte_centro.shape)

# 3. Descuento entre 10 y 20 (incluidos) con between
descuento_medio = ventas[ventas["descuento_pct"].between(10, 20)]
print(descuento_medio.shape)

# 4. Tres condiciones combinadas
combinado = ventas[
    (ventas["categoria"] == "Electronica")
    & (ventas["unidades"] > 2)
    & (ventas["descuento_pct"] != 0)
]
print(combinado.shape)

# 5. Invertir un filtro con ~ (los que NO son de Oficina)
no_oficina = ventas[~(ventas["categoria"] == "Oficina")]
print(no_oficina.shape)

# 6. Mismo filtro del punto 1 con .query()
online_altos_query = ventas.query("canal == 'Online' and importe_netto > 500")
print(online_altos_query.shape)
# .query() es más legible porque se escribe casi como una frase,
# sin tener que repetir "ventas[...]" ni usar & / | en vez de and / or

# 7. Empleados de Ingeniería, remotos, con más de 5 años, ordenados por salario
ingenieria_remoto_senior = empleados[
    (empleados["departamento"] == "Ingenieria")
    & (empleados["modalidad"] == "Remoto")
    & (empleados["antiguedad_anios"] > 5)
].sort_values("salario_base", ascending=False)

print(ingenieria_remoto_senior)

# 8. Exportar el resultado
ingenieria_remoto_senior.to_csv("../outputs/ingenieria_remoto_senior.csv")

```

Explicación “Para Tontos”: - Cada condición individual (`ventas["canal"] == "Online"`) genera una columna de True/False; para combinarlas con `&` (Y) u `|` (O) hay que meter **cada condición entre paréntesis**, porque si no Python intenta evaluar `&` antes que `==` y da un resultado incorrecto o un error. - `.isin(["Norte", "Centro"])` comprueba, para cada fila, si el valor de esa columna **está dentro de la lista** indicada — mucho más cómodo que encadenar varios `==` ("Norte") | (... == "Centro") - `.between(10, 20)` comprueba que el valor esté **dentro de un rango**, incluyendo ambos extremos — equivale a escribir `(columna >= 10) & (columna <= 20)` pero de forma más compacta. - `~` es el operador de **negación** para condiciones de pandas (equivalente a `not`, pero para Series booleanas): `~(condición)` se queda con las filas donde la condición **NO** se cumple. - `.query("expresión en texto")` permite escribir el filtro como si fuera una frase, usando directamente `and/or/==` dentro de las comillas en vez de `&/|` fuera de ellas — el resultado es exactamente el mismo que con corchetes, pero para muchas condiciones a la vez suele leerse más claro. - Para el filtro de empleados se encadenan **tres condiciones** con `&` (departamento, modalidad y antigüedad) y al final se aplica `.sort_values("salario_base", ascending=False)` para ordenar el resultado filtrado de mayor a menor salario, todo en una única expresión.

Sección 8 — Bucles

Ejercicio 19 — Bucles sobre estructuras de datos

Enunciado: > `while` para reposición de stock; `for` sobre lista en mayúsculas; `enumerate` con `start=1`; recorrido de lista de listas desempaquetando; recorrido de diccionario; recorrido de array NumPy; y por qué hace falta `.items()` en un diccionario.

Código en Python:

```
# 1. while: reposición de stock
stock = 12
lote = 0
while stock < 60:
    lote = lote + 1
    stock = stock + 8
    print("Lote", lote, "-> stock:", stock)

# 2. for sobre una lista, en mayúsculas
regiones = ["Norte", "Sur", "Este", "Oeste", "Centro"]
for region in regiones:
    print(region.upper())

# 3. enumerate empezando en 1
for indice, region in enumerate(regiones, start=1):
    print(f"Región {indice}: {region}")

# 4. Lista de listas, desempaquetando cada fila
inventario = [
    ["ALM-NORTE", "Robot Aspirador", 34, 201.50],
    ["ALM-NORTE", "Monitor 27", 58, 142.00],
    ["ALM-SUR", "Silla Ergonomica", 12, 178.90],
]
for almacen, producto, unidades, coste in inventario:
    print(almacen, "|", producto, "|", round(unidades * coste, 2), "€")

# 5. Diccionario con .items()
tarifas_envio = {"Norte": 4.95, "Sur": 5.50, "Este": 5.20}
for region, tarifa in tarifas_envio.items():
    print(region, "->", tarifa)

# 6. Array de NumPy
import numpy as np
np_precios = np.array([59.90, 279.00, 18.50])
for precio in np_precios:
    print(round(precio * 1.04, 2))
```

Explicación “Para Tontos”: - En el `while`, la variable `stock` empieza en 12 y se le van sumando **lotes de 8 unidades** en cada vuelta, mientras siga siendo menor que 60; `lote` cuenta cuántas vueltas van dadas. El bucle se detiene automáticamente en cuanto `stock` llega a 60 o más — esto es estilo **imperativo** porque depende de variables mutables (`stock`, `lote`) que cambian paso a paso. - El `for region in regiones:` recorre la lista **elemento a elemento**, sin necesidad de índice ni contador; `region.upper()` convierte cada nombre a mayúsculas sobre la marcha. - `enumerate(regiones, start=1)` da, en cada vuelta, tanto la **posición** como el **valor**, y `start=1` hace que la numeración empiece en 1 en vez de en 0 (más natural para mostrarlo a un humano, “Región 1” en vez de “Región 0”). - En `for almacen, producto, unidades, coste in inventario:`, cada fila de la lista de listas se **desempaqueta automáticamente** en cuatro variables en la propia cabecera del bucle, evitando tener que escribir `fila[0]`, `fila[1]`... dentro del cuerpo. - **Punto 7 (respuesta):** al recorrer un diccionario, si solo escribes `for x in diccionario:` obtienes **únicamente las claves** (no los valores); para obtener clave y valor a la vez en cada vuelta hace falta llamar explícitamente a `.items()`, que devuelve pares (`clave`, `valor`) que se pueden desempaquetar directamente en la cabecera del `for`. - Recorrer un array de NumPy con un `for` normal funciona igual que con una lista, aunque para operaciones sobre **todo el array** siempre es preferible la versión vectorizada (`np_precios * 1.04`) en vez de un bucle, por rapidez.

Ejercicio 20 — Iteración sobre DataFrames

Enunciado: > Recorre las 5 primeras filas con `.iterrows()`; crea `codigo_region` con el accesor `.str`; crea `segmento_ticket` con `.apply()` sobre `clasificar_ticket`; cuenta con `.value_counts()`; crea `eficiencia` sin bucle; construye `resumen_por_region` con un `for`; y compáralo con el `.groupby()` del ejercicio 15.

Código en Python:

```

import pandas as pd
import sys
sys.path.append("../src")
from utilidades import clasificar_ticket

ventas = pd.read_csv("../data/ventas_retail.csv")
ventas["importe_bruto"] = ventas["unidades"] * ventas["precio_unitario"]
ventas["importe_netto"] = round(ventas["importe_bruto"] * (1 - ventas["descuento_pct"] / 100), 2)
sensores = pd.read_csv("../data/sensores_planta.csv")

# 1. Recorrer las 5 primeras filas con iterrows
for etiqueta, fila in ventas.head(5).iterrows():
    print(fila["region"], "|", fila["producto"], "|", fila["importe_netto"], "€")

# 2. Código de región con el accesor de texto .str
ventas["codigo_region"] = ventas["region"].str[:3].str.upper()
print(ventas[["region", "codigo_region"]].head())

# 3. Segmento de ticket con apply, usando la función importada
ventas["segmento_ticket"] = ventas["importe_netto"].apply(clasificar_ticket)

# 4. Contar pedidos por segmento
print(ventas["segmento_ticket"].value_counts())

# 5. Columna eficiencia, sin bucle (operación vectorizada de pandas)
sensores["eficiencia"] = (sensores["unidades_producidas"] / sensores["temperatura_c"]).round(3)
print(sensores[["id_maquina", "unidades_producidas", "temperatura_c", "eficiencia"]].head())

# 6. Diccionario resumen_por_region construido con un bucle for
resumen_por_region = {}
for region in ventas["region"].unique():
    filtrado = ventas[ventas["region"] == region]
    resumen_por_region[region] = round(filtrado["importe_netto"].sum(), 2)

print(resumen_por_region)

# 7. Comparación con el resultado de groupby del ejercicio 15
resumen_groupby = ventas.groupby("region")["importe_netto"].sum().round(2)
print(resumen_groupby)

```

Explicación “Para Tontos”: - `.iterrows()` recorre el DataFrame **fila a fila**, devolviendo en cada vuelta la etiqueta del índice (`etiqueta`) y la fila entera como una Serie (`fila`), a la que se accede por nombre de columna (`fila["region"]`). Es útil para entender el concepto de “recorrer filas”, pero en la práctica suele ser **más lento** que usar operaciones vectorizadas (como en el punto 5), así que se reserva para casos donde de verdad haga falta procesar cada fila de forma independiente. - `.str` es un **accesor especial** de pandas para trabajar con columnas de texto igual que si fueran strings normales de Python: `.str[:3]` hace slicing de los tres primeros caracteres de **cada valor de la columna a la vez**, y `.str.upper()` los pasa a mayúsculas — sin necesidad de ningún bucle. - `ventas["importe_netto"].apply(clasificar_ticket)` aplica la función importada desde `src/utilidades.py` a cada valor de la columna; igual que en el ejercicio 17, se pasa `clasificar_ticket` **sin paréntesis**, porque `.apply()` es quien se encarga de llamarla por ti sobre cada valor. - El cálculo de `eficiencia` en el punto 5 se hace dividiendo **directamente dos columnas** (`sensores["unidades_producidas"] / sensores["temperatura_c"]`); pandas aplica la división fila a fila automáticamente, y `.round(3)` redondea el resultado completo a tres decimales — no hace falta ningún bucle porque es una operación vectorizada, igual que las de NumPy. - `ventas["region"].unique()` devuelve la lista de **valores distintos** que aparecen en esa columna, sin repetidos; el bucle `for region in ...` recorre esas regiones una a una, filtra el DataFrame para quedarse solo con las filas de esa región (`ventas[ventas["region"] == region]`), suma su `importe_netto`, y guarda el resultado en el diccionario `resumen_por_region` usando la región como clave. - Al comparar `resumen_por_region` (construido “a mano” con un bucle `for`) con el resultado de `.groupby("region")["importe_netto"].sum()` del ejercicio 15, ambos deberían dar **exactamente las mismas cifras** — son dos formas distintas de llegar al mismo resultado. La versión con `.groupby()` es preferible en la práctica: es más corta, más rápida (está optimizada internamente) y menos propensa a errores que escribir el bucle a mano; la versión manual sirve sobre todo para entender **qué está haciendo** `.groupby()` por debajo.